

The Collatz Conjecture at the Critical Line: A Friendly Guide

Companion notes to the paper of the same title*

June 12, 2026

1 The game

Take any positive whole number. If it is even, halve it. If it is odd, multiply by three and add one. Repeat.

$7 \rightarrow 22 \rightarrow 11 \rightarrow 34 \rightarrow 17 \rightarrow 52 \rightarrow 26 \rightarrow 13 \rightarrow 40 \rightarrow 20 \rightarrow 10 \rightarrow 5 \rightarrow 16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1.$

The **Collatz conjecture** says: no matter which number you start with, you always end at 1. Computers have checked every starting number up to about 2.4 sextillion (2^{71}). Nobody has proved it always happens, and nobody has found a number that escapes. This guide explains a collection of theorems — each checked line-by-line by a computer proof assistant called Lean, so there is no “trust me” anywhere — that map out exactly *why* the problem is hard and *how close* mathematics can currently get.

One bookkeeping convenience first. When n is odd, $3n + 1$ is always even, so the next move is always a halving. It saves ink to fuse them: the *Terras map* sends odd n to $(3n + 1)/2$ and even n to $n/2$. Every step now includes exactly one halving, which makes size accounting clean. “Odd step” (roughly multiply by 1.5) versus “even step” (multiply by 0.5) is the whole tug-of-war.

2 Why it should be true — and why that’s not a proof

Here is the back-of-envelope argument every mathematician meets first. If the odd/even choices behaved like fair coin flips, a typical step would multiply your number by about $\sqrt{1.5 \times 0.5} = \sqrt{0.75} \approx 0.87$ — shrinking. So a “random” orbit drifts down and eventually hits 1. The catch: orbits are not random. They are completely determined by the starting number. The conjecture is a claim about *every individual* number, and averages say nothing about the worst case. The gap between “almost every number behaves” and “every number behaves” is, as we will see, precisely where the whole problem lives.

*Written in collaboration with Claude (Anthropic). These notes explain the ideas; the paper and the Lean files contain the proofs. Nothing here is a proof, and nothing here claims the conjecture — it remains open.

The critical ratio. Suppose the first T steps contain j odd steps. Up to small corrections your number has been multiplied by $3^j/2^T$. That exceeds 1 exactly when j/T exceeds

$$\frac{\log 2}{\log 3} = 0.6309\dots$$

So: an orbit climbs while more than about 63% of its steps are odd, and falls otherwise. Everything in this story orbits around that one number, the *critical density*.

3 The tempting proof strategy, and the theorem that kills it

Because any finite range can be checked by computer, the most natural plan is:

1. Sort numbers into finitely many bins — say, by remainder when dividing by 2^k .
2. Check, bin by bin, that every number is guaranteed to drop below its starting value within some fixed number of steps.
3. Conclude every large number eventually falls into the already-verified range, and you are done.

Step 3 is fine. Step 2 *feels* like a finite computation, and countless attempted proofs rest on some version of it. The first main theorem says step 2 is not just unproven but **impossible**: no binning, no clever weighting of bins, no fixed step budget can ever work.

The villain. Consider $n = 2^L - 1$: in binary, a string of L ones. It is odd, so apply $(3n + 1)/2$. A short calculation gives $3 \cdot 2^{L-1} - 1$ — whose binary form ends in $L - 1$ ones. Still odd. Apply again: ends in $L - 2$ ones. The number climbs by a factor of about 1.5 per step for L consecutive steps: 100% odd density, far above critical, for as long as you arranged ones in advance.

Why does this defeat *every* binning argument? A bin “knows” only a bounded amount about a number — its remainder, the state of some finite gadget, a bounded score. But $2^L - 1$ exists for every L . Whatever the bound, choose L so large that the all-ones number climbs straight through any fixed step budget; the bookkeeping that was supposed to certify descent gets exhausted. (In fancier language: -1 behaves like a fixed point of the odd step — check: $(3 \cdot (-1) + 1)/2 = -1$ — and $2^L - 1$ imitates -1 for L steps. The fancier language is what the formal proof uses, but the binary picture is the same fact.)

The no-go theorem. Any descent certificate whose memory of n is bounded — remainder classes for any modulus, any finite automaton, any bounded weighting, any fixed window of steps — is defeated by the numbers $2^L - 1$. This closes off an entire genre of proof attempts at once, including weighted and amortized versions.

A concrete sample, verified in Lean down to the witness: the plausible claim “every $n > 300$ drops below itself within 53 steps” is false, and the smallest counterexample is 447 — binary 110111111, whose six trailing ones are a pocket-sized version of the villain.

4 The information budget: why the first $\log_2 n$ steps are special

Here is the cleanest way to understand both the obstruction and the positive results. When you halve a number, you consume its last binary digit. A number with $\log_2 n$ digits can therefore only “answer questions” about its first $\log_2 n$ halvings; beyond that, the digits are spent and the orbit’s choices are fresh information you never had.

This intuition is a theorem (Terras, 1976, formalized here):

- the pattern of odd/even choices in the first k steps is *exactly determined* by the remainder of n upon division by 2^k — and conversely, the pattern determines the remainder;
- consequently **every** odd/even pattern of length k is realized by exactly one remainder class. All behaviors occur. The all-ones villain is just one cell of a complete table;
- counting cells: the number of remainder classes whose first k steps contain exactly j odd ones is the binomial coefficient $\binom{k}{j}$ — Pascal’s triangle, the same count as coin-flip sequences.

Why the problem is genuinely two-faced. Every finite behavior occurs (so worst-case reasoning is hopeless — that is the no-go theorem), yet behaviors are distributed like coin flips (so typical numbers fall — that is where the “almost all” results come from).

5 The corridor: exact bookkeeping

The heuristic “multiplied by $3^j/2^T$ ” can be made exact, with the error terms under complete control. Two inequalities, proved by induction with nothing but integer arithmetic, sandwich the truth:

$$3^j(n+1) - 2^j \leq 2^T \cdot (\text{value after } T \text{ steps}) \leq 3^j(n + 2^{T-j}) - 2^T.$$

Both become exact equalities on the villain orbit, so neither can be improved. From the sandwich follow the two walls of the corridor:

Wall 1 (forced density). If n has not yet dropped below its starting value after T steps, then either its odd-step share already exceeds the critical 63.09%, or it has used up more than $\log_2 n$ halvings — its entire information budget. Inside the budget, survival *requires* supercritical density. No exceptions.

Wall 2 (the wall is touched). Conversely, for every window length B there really are arbitrarily large numbers that survive all B steps with odd density only a hair above 63.09% — one can prescribe the pattern “ j odd steps then s even ones” and, by the pattern-realization theorem, manufacture actual integers that follow it. The achievable survival densities form a strip of width about $1/B$ sitting exactly on the critical line, inhabited at every scale.

And a reduction theorem ties the knot: the full conjecture is *equivalent* to the statement that every $n \geq 2$ eventually drops below itself. So:

What the conjecture actually is. A number escapes to infinity only by keeping its odd-step share above 63.09% forever — threading a strip that narrows like $1/B$ at every scale simultaneously. The strip is never empty at any finite scale, and no bounded-memory argument can decide who threads it. That tension *is* the Collatz problem.

6 Almost every number falls (and it’s elementary)

Combine the walls with Pascal’s triangle. To survive k steps a number needs a supercritical pattern, but among the 2^k remainder classes only $\sum_{j \gtrsim 0.63k} \binom{k}{j}$ have one — an exponentially small minority, since coin flips concentrate near 50%. Conclusion (Terras’ theorem, here in fully finite form): among the numbers between 2^k and 2^{k+1} , the fraction that fails to drop within k steps is at most about $2^{-0.045k}$ — vanishing exponentially.

Two touches make the formal version pleasant. The binomial tail is tamed by a trick worth showing off: multiply each tail term by $2^j/2^m \geq 1$ and use

$$2^m \sum_{j \geq m} \binom{k}{j} \leq \sum_j \binom{k}{j} 2^j = (1 + 2)^k = 3^k,$$

the binomial theorem — no calculus, no entropy, nothing beyond algebra. And the threshold “63.09%” is replaced by the fraction $17/27 = 0.6296\dots$, justified by the single inequality $3^{17} \leq 2^{27}$. The price of staying this elementary is a slightly generous constant; the prize is that a computer can certify every step.

7 Why there are no small loops

A number could also fail by entering a loop (other than the familiar $1 \rightarrow 2 \rightarrow 1$). Going once around a loop of T steps with j odd ones multiplies by almost exactly $3^j/2^T$ — and must return you to where you started, so $3^j/2^T \approx 1$, i.e. $3^j \approx 2^T$. But a power of 3 never *equals* a power of 2, and near-misses are governed by how well the irrational number $\log_2 3$ can be approximated by fractions T/j — which is “badly,” most of the time. The sandwich inequalities turn this into a clean bound: every member of a loop is smaller than $3^j 2^{T-j}/(2^T - 3^j)$.

Feed in the computer-verified fact that everything below 2^{71} reaches 1, and a finite check (performed by Lean’s proof kernel itself, all 17,000 or so cases) yields: **no loop with 183 or fewer halvings exists**. The first case the method cannot kill is $T = 184$, $j = 116$, where 3^{116} approximates 2^{184} just well enough — the wall is a genuine number-theoretic fact, not a software limitation.

8 Beyond the theorem: the experimental frontier

The no-go theorem forbids *bounded*-memory certificates. What about scores that read *all* the binary digits of n — for instance $V(n) = n \cdot c^{(\text{number of ones in } n)}$? These genuinely escape the theorem’s hypotheses, and on the villain they work: the all-ones orbit loses ones as it climbs, so for $c > 9/4$ the score falls even while the number rises. The papers report a systematic

machine search (with the search’s own counterexample-finder) over such digit-reading scores. Every candidate died, and the autopsies found three recurring killers:

1. **Injection:** start from a number whose digits make the score artificially tiny (lots of zeros); the dynamics restores typical digit statistics and the score balloons — even while the number itself shrinks.
2. **Global rewrite:** an odd step multiplies the whole binary string by 3; digit counts get scrambled non-locally, so no digit-based score is steered by the digits it read.
3. **Shift release:** a single halving shifts the whole string, instantly cashing in whatever “stored credit” a position-sensitive score had accumulated — the best candidate found (it effectively predicted the villain’s climbs in advance) died this way in one step.

The pattern of the three killers points somewhere specific: binary (base-2) statistics are destroyed by multiplication by 3, and base-3 statistics would be destroyed by halving. Anything that survives both must live on base 2 and base 3 *simultaneously* — and the interaction of doubling and tripling is a famously deep area of mathematics (Furstenberg’s “ $\times 2, \times 3$ ” problems). The experiments, in other words, independently rediscover where the experts always suspected the difficulty lives.

9 What “machine-verified” means here

Every theorem above is written in Lean 4, a proof assistant whose small trusted kernel checks each logical step; “the computer checked it” means checked the *proof*, not merely tested examples. The files compile with no unproven placeholders, and an audit confirms no axioms beyond Lean’s standard foundations. The one statement taken on faith is clearly labeled as a hypothesis: that computers have verified the conjecture below 2^{71} (a published computation, not formalized).

10 Summary in five sentences

Numbers fall on average but the conjecture is about every number. Any proof that remembers only finitely much about n is provably impossible, because all-ones numbers climb past any budget. Each number controls only its first $\log_2 n$ steps, behaviors within that budget are distributed like coin flips, and survival demands an exponentially rare 63%-odd pattern — so almost every number falls, provably and elementarily. Escape would mean riding the 63.09% critical line at every scale forever, through a strip that narrows like $1/B$ yet is never empty; loops would need $3^j \approx 2^T$ and are ruled out up to 183 halvings. What remains open is exactly the gap between “almost every” and “every” — and that gap is now fenced, measured, and machine-checked on both sides.